AI Shopping Assistant - API Documentation

This document describes all backend API endpoints, request/response formats, and how the frontend integrates with them.

Base URL

- **Development**: `http://localhost:8000/api`
- **Production**: Proxied through Express server at `/api`

Authentication

The API uses a simple customer ID + password authentication system. After login, the customer ID is used to identify the user in subsequent requests.

Endpoints

Health Check

`GET /api/health`

Check if the API server is running.

Response:

```
{
  "status": "healthy"
}
```

Authentication

`POST /api/login`

Authenticate a customer with their ID and password.

Request Body:

```
{
  "customer_id": "CUST-0000001",
  "password": "password123"
}
```

Response (Success):

```
{
  "success": true,
  "customer": {
    "customer_id": "CUST-0000001",
    "first_name": "John",
    "last_name": "Doe",
    "email": "john.doe@email.com"
  }
}
```

Response (Failure):

```
{
  "success": false,
  "message": "Invalid password"
}
```

Customer Endpoints

`GET /api/greeting/{customer_id}`

Get a personalized greeting for the customer.

Parameters: - `customer_id` (path): Customer ID (e.g., "CUST-0000001")

Response:

```
{
  "greeting": "Welcome back, John! How can I help you today?",
```

```
  "customer_name": "John"
}
```

`GET /api/customer360/{customer_id}`

Get comprehensive customer profile including preferences, purchase history, and style profile.

Parameters: - `customer_id` (path): Customer ID

Response:

```
{
  "customer_id": "CUST-0000001",
  "first_name": "John",
  "last_name": "Doe",
  "email": "john.doe@email.com",
  "preferences": {
    "favorite_brands": ["Nike", "Adidas"],
    "preferred_styles": ["casual", "athletic"]
  },
  "style_profile": {
    "color_preferences": ["blue", "black"]
  },
  "size_info": {
    "shirt": "M",
    "pants": "32"
  }
}
```

`GET /api/customers/{customer_id}`

Get basic customer information.

`PUT /api/customers/{customer_id}`

Update customer information.

Request Body:

```
{
  "name": "John Doe",
  "email": "john.doe@email.com",
  "preferences": {},
  "style_profile": {},
  "size_info": {}
}
```

Chat / AI Assistant

`POST /api/chat`

Send a message to the AI shopping assistant and receive a response with product recommendations.

Request Body:

```
{
  "message": "I'm traveling to Paris next week",
  "user_id": 1,
  "conversation_id": null
}
```

Response:

```
{
  "response": "How exciting! Paris in winter is beautiful. What activities do you have planned?",
  "products": [
    {
      "id": 1,
      "name": "Classic Wool Coat",
      "description": "Elegant wool coat perfect for Paris weather",
      "category": "Outerwear",
      "subcategory": "Coats",
      "price": 299.99,
      "brand": "Burberry",
      "gender": "unisex",
      "image_url": "/images/coat.jpg",
      "in_stock": true,
      "rating": 4.5,
      "sizes_available": ["S", "M", "L", "XL"]
    }
  ],
  "clarification_needed": false,
  "clarification_question": null,
  "context": {
    "intent": {
      "destination": "Paris, France",
      "travel_date": "2026-02-01",
      "activities": ["sightseeing"]
    },
    "environmental": {
      "weather": {
        "temperature": 8,
        "condition": "partly cloudy"
      }
    }
```

```
  },
  "updated_intent": {
    "destination": "Paris, France",
    "destination_city": "Paris",
    "destination_country": "France",
    "travel_date": "2026-02-01",
    "activities": null,
    "clothes": null
  },
  "agent_thinking": [
    {
      "agent": "Clarifier",
      "action": "Analyzing user intent",
      "details": {"destination_detected": "Paris"},
      "timestamp": "2026-01-28T10:00:00Z"
    }
  ],
  "suggestions": [
    "Show me winter coats",
    "What about formal wear?",
    "Beach activities",
    "Business meetings"
  ]
}
```

Key Fields: - `response`: The assistant's text response - `products`: Array of recommended products - `suggestions`: Quick-reply bubble suggestions for the user - `context`: Current conversation context (intent, weather, etc.) - `updated_intent`: The parsed shopping intent from the conversation - `agent_thinking`: Debug info showing agent reasoning steps

---

Conversation Management

`GET /api/conversation/{user_id}`

Retrieve the conversation history for a user.

Parameters: - `user_id` (path): User's numeric ID

Response:

```
{
  "messages": [
    {"role": "user", "content": "I'm traveling to Paris"},
    {"role": "assistant", "content": "How exciting! When are you going?", "products": []}
  ],
  "context": {
    "destination": "Paris, France"
```

```
  }
}
```

`POST /api/reset`

Reset/clear the conversation history for a user.

Query Parameters: - `user_id` (required): User's numeric ID (integer)

Example:

```
POST /api/reset?user_id=1
```

Response:

```
{
  "status": "conversation reset"
}
```

Products

`GET /api/products`

Get all products from the catalog.

Response:

```
[
  {
    "id": 1,
    "name": "Classic Wool Coat",
    "description": "Elegant wool coat",
    "category": "Outerwear",
    "subcategory": "Coats",
    "price": 299.99,
    "brand": "Burberry",
    "gender": "unisex",
    "sizes_available": ["S", "M", "L"],
    "colors": ["black", "navy"],
    "image_url": "/images/coat.jpg",
    "in_stock": true,
    "rating": 4.5
  }
]
```

`GET /api/products/{product_id}`

Get a single product by ID.

Frontend Integration

API Client Setup

The frontend uses TanStack React Query for data fetching. API calls are made through a proxy at `/api` which forwards to the Python backend.

```
// Example: Sending a chat message
const chatMutation = useMutation({
  mutationFn: async (message: string) => {
    const response = await fetch("/api/chat", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        message,
        user_id: userId,
      }),
    });
    return response.json();
  },
  onSuccess: (data) => {
    // Handle response with data.response, data.products, data.suggestions
  }
});
```

Key Integration Points

1. **Login Flow**: `POST /api/login` → Store customer info in state
2. **Chat Interaction**: `POST /api/chat` → Display response, products, and suggestions
3. **Customer Profile**: `GET /api/customer360/{id}` → Show personalized greeting
4. **Conversation Persistence**: `GET /api/conversation/{id}` → Restore chat history
5. **Reset Conversation**: `POST /api/reset` → Clear history for new trip

Response Handling

```
interface ChatApiResponse {
  response: string;
```

```
  products: Product[];
  suggestions: string[];
  context: {
    intent: NormalizedIntent;
    environmental: EnvironmentalContext;
  };
  updated_intent: ClarifierIntent;
  agent_thinking: AgentThinkingStep[];
}
```

Error Handling

All endpoints return appropriate HTTP status codes:

- `200` : Success
- `400` : Bad Request (invalid input)
- `404` : Not Found (resource doesn't exist)
- `500` : Internal Server Error

Error responses include a `detail` field with the error message:

```
{
  "detail": "Customer not found"
}
```